

Assessment 2B: Case Study Implementation

UO System Design and Realisation
INFO 2026

Manny Asbanu
a1926920

Class Level Diagram

See attached PDF: 2A-class-diagram.pdf

Design Changes

The final class structure, interfaces, dependencies, and component responsibilities, were unchanged from the Assessment 2A design. No classes or component boundaries were added, removed, or reassigned.

During implementation, the *total_minutes* calculation was changed. Assessment 2A treated *duration_min* as a value to add for every selected paper. However, this value belongs to the complete session and not an individual paper. For example, if two selected papers are in the same 60 minute session, counting the duration for each paper would incorrectly produce 120 minutes. The final implementation counts each distinct session once instead, giving a total of 60 minutes. Different sessions are counted separately, even when their times overlap, because the summary reports the total duration of the sessions covered rather than elapsed time free of clashes. This changed only the calculation, so the class structure and public summary format remained unchanged.

Implementation Reflection

Development Process

The implementation was completed incrementally, first programme import and retrieval followed by schedule building, comparison, joint schedule saving, and summary generation. The class

diagram and façade contracts were used to define responsibilities before each component was implemented. This kept the implementation aligned with the original design.

Codex was used to assist implementation through syntax verification and some code generation. This was based on my design, and the required classes, dependencies, behaviour, error handling, and scope was defined during prompting to ensure Codex did not design autonomously. I reviewed all outputs and verified implementation through pytest. Testing and implementation were done together and failures helped identify incorrect behaviour and integration issues. Following supervisor feedback, the tests were also renamed and grouped by programme handling, schedule building, comparison, summaries, and complete workflows rather than UC number.

Correctness and Verification

Correctness is enforced inside each component. *CsvProgrammeImporter* validates input before *SessionAccess* replaces its catalogue, preventing failed imports leaving partially updated data. *ScheduleAccess* validates schedule names and paper titles before saving complete schedules. *Schedule* prevents duplicate papers while preserving their insertion order.

ScheduleComparator generates deterministic shared, unique, and merged results. Merged schedules contain papers from the first schedule followed by previously unseen papers from the second. *ScheduleService.save_joint_schedule()* leaves comparison to *ScheduleComparator* and storage to *ScheduleAccess*, avoiding duplicate comparisons. A joint schedule can be retrieved after it is saved, compared again, and summarised like a regular schedule.

SummaryGenerator counts selected papers by track and calculates scheduled time. If multiple papers belong to the same session, that session's duration is counted once. Different sessions with overlapping times are counted, this is because clash handling is outside the implemented requirements.

The test suite covers normal, exceptional, and boundary behaviour. This includes malformed and empty CSV files, duplicate titles, invalid names, unknown papers, empty schedules, identical and disjoint comparisons, overlapping sessions, atomic failures, and joint schedules. Workflow tests cover programme import, schedule building, schedule comparison, joint schedule saving, and summarising. Tests that delete the CSV after import verify later operations use internal storage.

Final Verification	Result
tests collected	82
tests passed	82

failures and errors	0
skipped failures	0
execution time	0.09 seconds

No coverage tool was used. Manual review confirmed every written class and public façade operation is used by component, integration, or workflow tests.

Testability

Component boundaries let the system be tested without the Flask interface. As *ScheduleComparator* and *SummaryGenerator* depend on access interfaces, behaviour can be kept separate using fake implementations. This avoids unnecessary CSV setup and keeps component tests focused.

Temporary CSV fixtures use real import paths where integration with programme data is required. Separate façade tests verify component wiring, while workflow tests verify the system through *ScheduleService*. Each test has one objective and a descriptive name.

Structure and Design Principles

Principle or pattern	Application	Effect
Single Responsibility Principle	Importing, programme access, schedule storage, comparison, and summary generation are handled by separate classes. <i>ScheduleService</i> performs coordination and result formatting.	Each class remains focused and can be changed or tested independently.
Dependency Inversion and Injection	<i>ScheduleComparator</i> and <i>SummaryGenerator</i> depend on <i>IScheduleAccess</i> and <i>ISessionAccess</i> . Dependencies are supplied through their constructors.	High level behaviour is separated from concrete storage, and tests can use small fake components.
Open/Closed Principle	Importing and data access are exposed through interfaces such as <i>IProgrammeImporter</i> , <i>ISessionAccess</i> , and <i>IScheduleAccess</i> .	Alternative import or storage implementations can be introduced without rewriting comparison or summary logic.

Façade	ScheduleService provides the single interface used by the supplied Flask application.	The interface remains independent of the internal component structure.
Command query separation	Loading, adding, and saving modify state, while listing, comparing, and summarising return information without modifying it.	Method behaviour is clearer and easier to test.

ScheduleService is the composition point and creates concrete components and supplies their dependencies. It does not store schedules, parse CSV data, or perform comparison and summary calculations. This keeps implementation consistent with class design and prevents the façade from having unrelated responsibilities.

Clean Code and Trade offs

Class, method, variable, and test names describe each purpose. Type hints explicitly label component contracts, and tuple returning queries prevent callers from modifying internal paper collections. Private validation and conversion helpers reduce duplication and do not add unnecessary layers. Comments are limited to decisions that are not obvious like atomic replacement and duration handling.

Because the application creates and reuses one *ScheduleService*, all schedules are stored in memory. This keeps implementation small and meets the internal storage requirement, however schedules are lost on application restart. Since consumers rely on *IScheduleAccess*, a database implementation could be incorporated without changing the comparator, summary generator, or façade contracts.